

UNIT STUDY NOTES

Data Visualization: Matplotlib, Seaborn and Plotly

A chart is an argument, not decoration. This unit covers building figures with Matplotlib, statistical graphics with Seaborn, and interactive output with Plotly, along with the design decisions that make a plot readable.

 6 source notebooks 112 topic sections extracted 6 study sections

Unit Overview

Topic: 08.01 · **Level:**  Intermediate · **Type:** THEORY + CODE

Welcome to Data Visualization

Everything you have learned until now — lists, loops, NumPy arrays and Pandas DataFrames — has been about *storing* and *manipulating* data. In this session we finally give that data a voice. Plotting turns thousands of numbers into a single picture your brain can read in a second, which is why data visualization is the secret weapon of every data analyst: it helps you spot trends, outliers, clusters and proportions long before any formal statistics does.

Matplotlib is the oldest and most widely used plotting library in Python. Its `pyplot` module, imported by convention as `plt`, gives you a simple stateful interface: draw a line with `plt.plot(x, y)`, points with `plt.scatter(x, y)`, bars with `plt.bar(...)`, a histogram with `plt.hist(...)` and a pie with `plt.pie(...)`. Every chart is built on the same idea — a *figure* (the canvas) holding one or more *axes* (the actual plot area).

Before writing any plotting code you should ask two questions. First, **what kind of data** do you have? Data is either *numerical* (continuous or discrete numbers such as runs scored or price) or *categorical* (labels or groups such as batsman names or countries). Second, **how many columns** are you studying? *Univariate analysis* studies a single column (e.g. a histogram of runs), while *bivariate analysis* studies two columns together (e.g. average vs strike-rate). Choosing the right chart depends on both answers.

Here is the roadmap for this session:

1. **2D line plots** — trends over a continuous axis such as time.
2. **Scatter plots** — the relationship between two numerical columns.
3. **Bar charts** — aggregates grouped by categories.
4. **Histograms** — the frequency distribution of one numerical column.
5. **Pie charts** — the proportion of a whole contributed by each part.
6. **Styles, saving and cleanup** — making figures look professional.

Throughout the session we plot real cricket data — run-scoring records of players like Virat Kohli, Rohit Sharma and Chris Gayle — so you can connect every concept to a concrete, relatable example. Let us fire up the library and look at our first chart type.

What/Why

Matplotlib is plotting library for static/interactive visualizations.

Core

Figures, axes, subplots. Supports line/bar/scatter/hist.

When to use

Exploratory data analysis, communication.

AI relevance

Model diagnostics, EDA.

Topic: 08.02 · **Level:** 🟡 Intermediate · **Type:** THEORY + CODE

Taking Matplotlib to the Next Level

In the previous session you mastered the basic chart types — line, scatter, bar, histogram and pie. This session goes deeper: you will learn how to encode a *third and even a fourth dimension* into a single plot, arrange several charts into one grid, annotate individual data points, and finally plot in three dimensions.

We begin with **colored scatter plots**. A normal scatter has two dimensions, x and y. By passing a numeric column through the `c=` argument and choosing a colormap with `cmap`, the color of each point becomes an extra variable — a cheap clustering peek that lets you see species or classes instantly. Hand-in-hand with this is **figure size control**: `plt.figure(figsize=(width, height))` sets the canvas in inches *before* drawing, which hugely improves readability when a scatter holds thousands of points.

Next we look at **annotations** — writing labels directly on the chart with `plt.text(x, y, label)` and drawing arrows to specific points with `plt.annotate(...)`. Combined with reference lines drawn by `plt.axhline(y)` and `plt.axvline(x)`, these turn a raw scatter into an insight-rich analysis: thresholds, averages and outliers all become visible at a glance.

The second half of the session is about layout and the third dimension. **Subplots** let you build multi-panel figures — either with `plt.subplots(nrows, ncols)` returning a whole array of axes at once, or with the manual `fig.add_subplot(row, col, index)` style. Then come **3D plots**: scatter, line, surface and contour, created by passing `projection='3d'` when making an axis. Using `np.meshgrid` and a function of two variables you can visualize surfaces like $z = x^2 + y^2$ or $z = \sin(x) + \cos(y)$.

Finally we touch on **heatmaps** with `plt.imshow()` (a matrix rendered as colored cells) and the extremely convenient **Pandas** `.plot(kind=...)` interface, which wraps Matplotlib so a single `df.plot(kind='pie')` gives you a chart in one line. By the end of the session you will be able to produce publication-ready, multi-dimensional and multi-panel figures for any analysis.

Topic: 08.03 · **Level:** ● Intermediate · **Type:** THEORY + CODE

Why Seaborn?

Matplotlib gave you pixel-level control — every line, marker and axis tuned by hand. Seaborn builds on top of Matplotlib and adds three things that make daily analysis dramatically easier: a **layer of abstraction** (complex statistical plots in one line), **better aesthetics** (beautiful default colors and styles) and **more graph types** (density, facet grids, heatmaps, cluster maps and more). If Matplotlib is a toolbox of raw parts, Seaborn is the pre-assembled furniture.

Seaborn functions come in two flavours, and knowing the difference is the single most important idea of the session:

- **Axes-level functions** (`sns.scatterplot`, `sns.lineplot`, `sns.histplot`, `sns.kdeplot`, `sns.boxplot`, `sns.heatmap` ...) draw onto the current Matplotlib axes and return that axes object. They give you fine control and compose naturally with `plt`. But they cannot create multi-panel figures by themselves.
- **Figure-level functions** (`sns.relplot`, `sns.displot`, `sns.catplot`, `sns.lmplot`, `sns.pairplot` ...) own an entire figure. They build a `FacetGrid` internally, so they support faceting with `col=` and `row=`, and they manage the legend automatically.

Seaborn's plots fall into a small number of families, the session's roadmap:

1. **Relational plots** — the statistical relationship between variables (`scatterplot`, `lineplot`).
2. **Distribution plots** — how values spread out (`histplot`, `kdeplot`, `rugplot`, plus bivariate densities).
3. **Matrix plots** — grids of values colored by magnitude (`heatmap`, `clustermap`).

The star feature everywhere is **semantic mapping**: arguments like `hue=`, `style=` and `size=` map columns onto color, markers and point sizes, letting one scatter carry four variables at once. In this session we use the built-in `tips` dataset (dinner bills and tips), Gapminder life-expectancy data from `plotly.express`, and the `titanic` and `iris` classics. Import `seaborn as sns`, and let's look at relational plots first.

Topic: 08.04 · **Level:** ● Intermediate · **Type:** THEORY + CODE

What this notebook covers

In Session 25 you met Seaborn's relational, distribution and matrix plot families. These all treat the data as points to scatter, densities to trace or grids to color. Part 2 tackles the plots you reach for when **one of your variables is categorical** (sex, day, smoker) — plus regression plots and the grid objects that let one figure hold many small panels. Everything runs on the built-in `tips` and `iris` datasets.

We start with **categorical plots**, which split into three groups. *Categorical scatter* — `stripplot`, `swarmplot` — puts every data point over a categorical axis; the trick is `jitter` (random horizontal offsets so overlapping points separate) or a beeswarm algorithm that packs points without overlap. *Categorical distribution* — `boxplot` and `violinplot` — summarize each category with quartiles or with a box-plus-density shape. *Categorical estimate* — `barplot`, `pointplot`, `countplot` — reduce each group to a statistic (mean, median, count) and draw bars or points with error bars. Because all of these use the same categorical x-axis, Seaborn

bundles them behind one figure-level function: `sns.catplot(kind='strip' | 'swarm' | 'box' | 'violin' | 'bar' | 'point' | 'count', ...)`. The golden rule: a categorical plot has a **categorical** x (or y) and usually a **numeric** y.

The notebook then moves to **regression plots**, where the goal is no longer description but modelling:

`regplot` draws a scatter plus the best-fit line and a 95% confidence band; `lmpplot` is the figure-level version with `hue` and faceting; `residplot` plots the residuals (distance between points and line) to check whether a straight line was a fair assumption.

Finally come the **grid objects** — multi-panel machinery. `FacetGrid` is a manual grid of panels arranged by `col`, `row` and `hue` onto which you `map` a plotting function. `PairGrid` creates a matrix of every numeric-variable pair, letting you choose separate plots for the diagonal (`map_diag`), upper (`map_upper`) and lower (`map_lower`) triangles — `pairplot` is its one-line default. `JointGrid` combines a 2D plot with univariate marginals on the edges — `jointplot` is its default. The notebook closes with `sns.get_dataset_names()` and `sns.load_dataset()`, your access to Seaborn's built-in datasets.

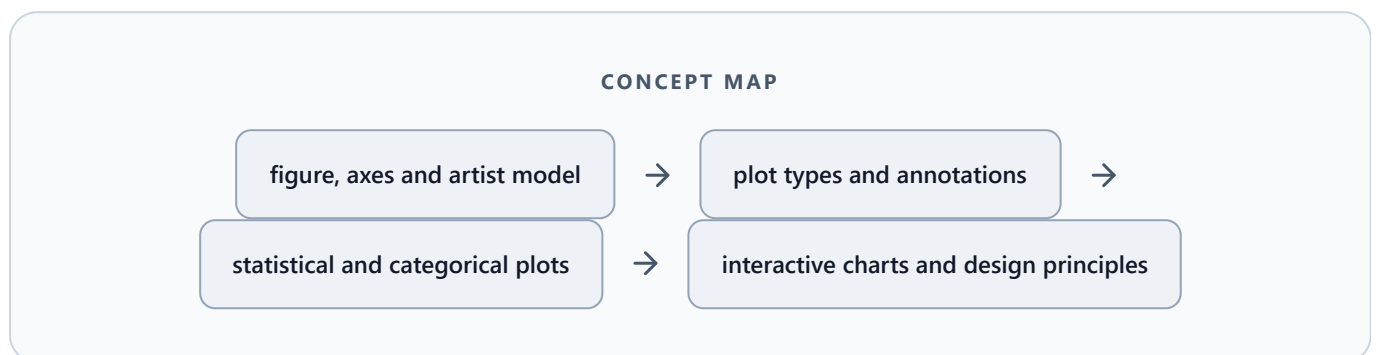
Prerequisites

→ Pandas

What you will be able to do

- ✓ Build and annotate figures directly in Matplotlib
- ✓ Use Seaborn for distributions, categories and relationships
- ✓ Add interactivity with Plotly where it earns its keep
- ✓ Choose encodings that do not mislead

What This Unit Covers



The sections below follow this arc, from foundational semantics to the engineering decisions that follow from them.

Core Concepts

Plotting with Matplotlib

Plotting with Matplotlib

Topic: 08.01 · **Level:**  Intermediate · **Type:** THEORY + CODE

Setup

- `matplotlib.pyplot` — the main module for plotting
- Convention: `plt`. Line, scatter, bar, hist, pie — everything comes from this
- Data types: Numerical vs Categorical (the analysis decides which one to use)
- Univariate (1 col) vs Bivariate (2 cols) — plan before you start

PYTHON

```
# import the library
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

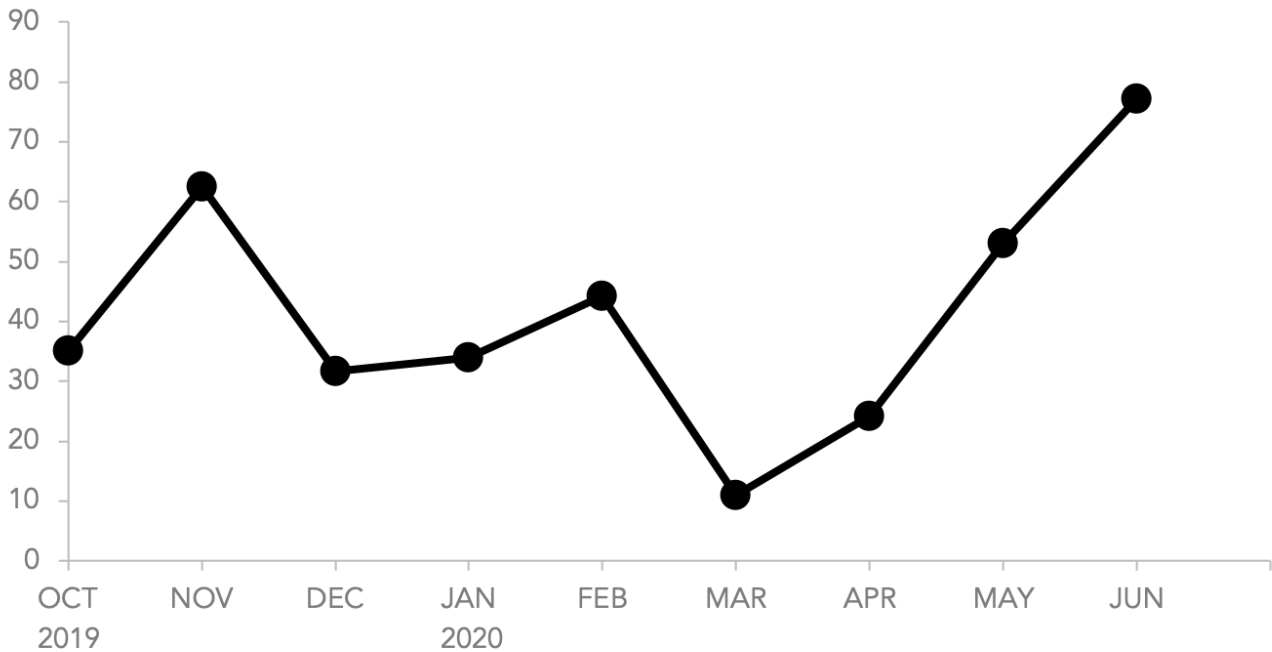
plt.style.use('default')
```

PYTHON

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

2D Line plot

Produce sales IN THOUSANDS (USD)



- **Bivariate** analysis — categorical/numerical × numerical
- From dataframe columns: `plt.plot(df['x'],df['y'])`
- Multiple plots — in the same figure: `plt.plot(a); plt.plot(b);plt.show()`
- Labels: `plt.xlabel()`, `plt.ylabel()`; Title: `plt.title()`
- Customization: `color`, `linestyle`, `linewidth`, `marker`, `markersize`
- Hex colors: `color='#D9F10F'`; legend loc: `plt.legend(loc=...)`
- Axis limits: `plt.xlim()`, `plt.ylim()`; grid: `plt.grid()`
- `plt.show()` — render at the end

PYTHON

```
plt.plot(x, y)
```

Pandas fundamentals

Labeled axes, alignment, indexing, grouping, missing data, I/O. Critical for EDA/preprocessing.

Deep Dive : 2D Line plot basics

- `plt.plot(x, y)` — basic 2D line
- $x \rightarrow$ x-axis, $y \rightarrow$ y-axis (lists/arrays/Series)
- Fig + axes auto-created; `plt.show()` renders
- Bivariate analysis: 2 numerical (or time→num) cols

- For DataFrames: `plt.plot(df['col1'], df['col2'])`
- Multiple plots: multiple `plt.plot()` before `show` — SAME axes, overlaid
- Customize each line independently
- Interactive vs inline backend differences notes
- Save / show / order matter
- Line = time trends & function graphs choice

PYTHON

```
# plotting a simple function
price = [48000, 54000, 57000, 49000, 47000, 45000]
year = [2015, 2016, 2017, 2018, 2019, 2020]

plt.plot(year, price)
```

PYTHON

```
# from a pandas dataframe
batsman = pd.read_csv(r"D:\GENAI\01_PREREQUISITS\00_Python\08_Data_Visualization\Datasets\sharma-kohli.csv")
batsman
```

Deep Dive : Labels, title, colors & style

- labels/title — context for the reader
- color: name 'red' / hex '#D9F10F' / rgb tuple
- linestyle: 'solid', 'dashed', 'dotted'
- linewidth, marker ('o', '+', '*', 's'), markersize
- legend — needs label= (or legend array) + loc
- xlim/ylim — limit/clamp axes (outliers!)
- grid — reference lines
- All customizations per-plot call
- Pixel-perfect control = granular matplotlib
- print available hex colors via docs

PYTHON

```
# Labels title
plt.plot(batsman['index'], batsman['V Kohli'])
plt.plot(batsman['index'], batsman['RG Sharma'])

plt.title('Rohit Sharma Vs Virat Kohli Career Comparison')
plt.xlabel('Season')
plt.ylabel('Runs Scored')
```

PYTHON

```
# colors(hex) and line(width and style) and marker(size)
plt.plot(batsman['index'],batsman['V Kohli'],color='#D9F10F')
plt.plot(batsman['index'],batsman['RG Sharma'],color='#FC00D6')

plt.title('Rohit Sharma Vs Virat Kohli Career Comparison')
plt.xlabel('Season')
plt.ylabel('Runs Scored')
```

Deep Dive : Figure show + styles + save

- show() — display (session notebooks auto-show sometimes)
- styles — preset rcParams themes: 'seaborn','ggplot','dark_background'
- savefig — save vector/raster: PNG, PDF, SVG
- dpi for quality; bbox_inches tight removes padding
- style applies to all subsequent figures
- custom rcParams — global tweaks
- Save BEFORE show (show resets!)
- Layout tools: figure, subplots later
- Style = brand look and matte consistency
- production charting pipeline ends savefig

PYTHON

```
# show
plt.plot(batsman['index'],batsman['V Kohli'],color='#D9F10F',linestyle='solid',linewidth=3,marker='D',mark
plt.plot(batsman['index'],batsman['RG Sharma'],color='#FC00D6',linestyle='dashdot',linewidth=2,marker='o')

plt.title('Rohit Sharma Vs Virat Kohli Career Comparison')
plt.xlabel('Season')
plt.ylabel('Runs Scored')

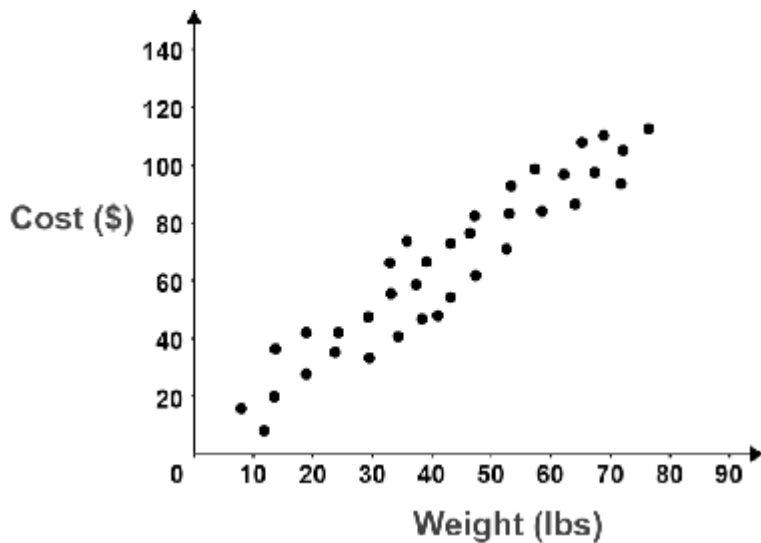
plt.grid()

plt.show()
```

PYTHON

```
plt.show() # render current figure
plt.style.available # built-in style names
plt.style.use('dark_background') # apply globally
plt.savefig('plots/plot.png', dpi=300, bbox_inches='tight')
```

Scatter Plots



- Bivariate Analysis
- numerical vs numerical
- Use case - Finding correlation
- **Bivariate** — numerical vs numerical
- Use case: to see correlation / relationship
- `marker` — point shape (`+`, `o`, etc.)
- `s` — point size (in scatter)
- `plt.plot(..., 'o')` — dots via style string, faster for large data
- Pass dataframe columns directly

PYTHON

```
plt.scatter(x, y, color='red', marker='+')
plt.plot(x, y, 'o')      # faster alternative
```

Deep Dive : Scatter plot — correlation

- `scatter(x, y)` — individual points
- Numerical vs numerical — checks correlation/pattern
- parameter marker (shape), s (size), c/color, alpha
- From DataFrame columns — data point clusters
- `plot(x,y,'o')` — same points but FASTER
- size mapping: s can be array → third dimension encoded!
- Use: outlier view, relationship shape, clusters
- Compare: scatterplot order matters repeated
- `plt.plot + marker` = fast alternative
- seaborn builds beautiful scatter later

PYTHON

```
# plt.scatter simple function
x = np.linspace(-10,10,50)

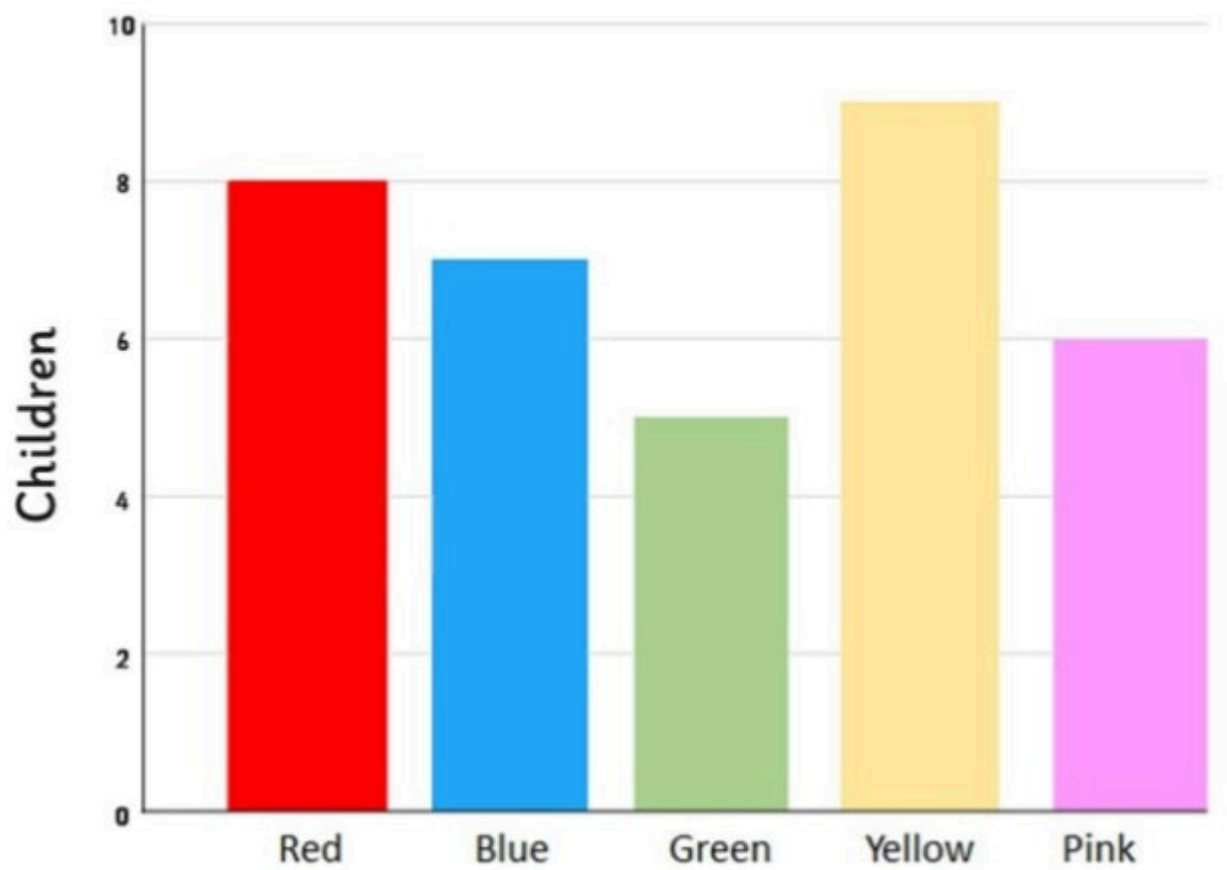
y = 10*x + 3 + np.random.randint(0,300,50)
y
```

PYTHON

```
plt.scatter(x,y)
```

Bar chart

Favourite Colour



- Bivariate Analysis
- Numerical vs Categorical
- Use case - Aggregate analysis of groups

Bar charts

- **Bivariate** — numerical vs categorical (aggregate)
- `barh` — horizontal bars

- Multiple bars: offset the x positions (`-0.2` , `+0.2`)
- Grouped/stacked: `plt.bar(df['b'],df['2017'],label='2017',bottom=...)`
- `xticks` , `label` — for the legend

PYTHON

```
plt.bar(x, height, width, color)
plt.barh(x, height) # horizontal
plt.bar(np.arange(n)-0.2, col1, width=0.2) # offset for grouped
```

Deep Dive : Bar charts — categorical

- bar — categorical vs numerical aggregates
- barh — horizontal bars (long labels fit)
- width — bar thickness
- multiple series — shift x positions (widths)
- stacked — second bar uses bottom=first
- color per bar via list
- xticks — custom tick labels
- Use: group comparisons/counts
- pandas `df.groupby(...).mean().plot` — quick agg
- bar = counts & category means one glance

PYTHON

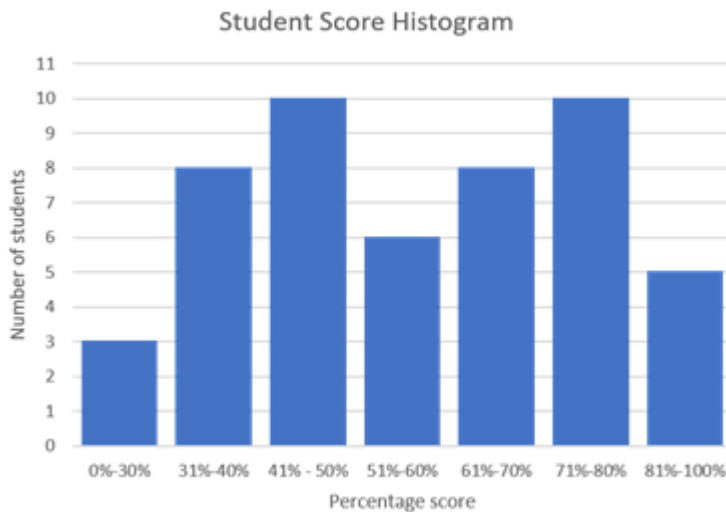
```
# simple bar chart
children = [10,20,40,10,30]
colors = ['red','blue','green','yellow','pink']

plt.bar(colors,children,color='black')
```

PYTHON

```
# horizontal bar chart
plt.barh(colors,children,color='black')
```

Histogram



- Univariate Analysis
- Numerical col
- Use case - Frequency Count

Pie charts

- **Univariate/Bivariate** — categorical vs numerical
- Use case: what portion of the whole each part represents (proportion)
- `autopct` — percentage labels
- `explode` — pull a slice out
- Only for rare categories (too many slices is a bad idea)

PYTHON

```
plt.pie(values, labels=..., autopct='%0.1f%%', explode=[...], shadow=True)
```

Histograms

- **Univariate** — numerical column
- Use case: frequency count / distribution
- `bins` — define ranges (`[0,10,20,...]`)
- Log scale for large arrays: `plt.yscale('log')`
- Distribution shape — spotting the mean/outliers

A histogram answers "what does one numeric column look like?" — data goes into bins (intervals) and each bin becomes a bar whose height is the count. In Plotly Express: `px.histogram(df, x='col', nbins=N)`. The `nbins` argument controls granularity (10 bins for 2007 life expectancy keeps the shape clean; 30 bins for iris sepal length shows the multi-modal detail), and `text_auto=True` prints the counts on top of each bar. Adding

`color='species'` separates the bars by category, showing you *three* overlaid histograms of the same variable. Histograms carry over the insight you got from seaborn's `displot`, but interactive: hover any bar to read its exact range and count.

PYTHON

```
plt.hist(data, bins=[10,25,40,55,70])
```

PYTHON

```
# histogram
# plot histogram of life expt of all countries in 2007 -> nbins -> text_auto
temp_df = gap[gap['year'] == 2007]

px.histogram(temp_df, x='lifeExp', nbins=10, text_auto=True)
```

Deep Dive : Histogram vs Pie vs Log scale

- Histogram — UNIVARIATE numerical frequency (bins)
- bins list → custom bucket edges; hist density option
- log scale — compressed skewed huge arrays
- Pie — categorical share of total
- autopct — percent format; explode slice; shadow decor; startangle rotate
- Pie weakness: compare small slices hard (bar better)
- use frequency & proportion diversity
- numpy loads .npy arrays for bulk hist
- Distribution shape (normal, skew) inspection
- Choose right viz for message type

PYTHON

```
# simple data

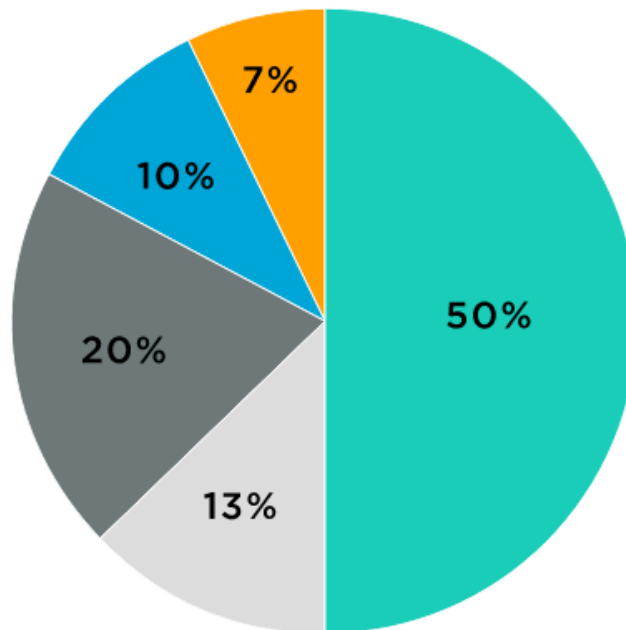
data = [32,45,56,10,15,27,61]

plt.hist(data,bins=[10,25,40,55,70])
```

PYTHON

```
# on some data
df = pd.read_csv(r"D:\GENAI\01_PREREQUISITS\00_Python\08_Data_Visualization\Datasets\vk.csv")
df
```

Pie Chart



- Univariate/Bivariate Analysis
- Categorical vs numerical
- Use case - To find contribution on a standard scale

PYTHON

```
# simple data
data = [23,45,100,20,49]
subjects = ['eng','science','maths','sst','hindi']
plt.pie(data,labels=subjects)

plt.show()
```

PYTHON

```
# dataset
df = pd.read_csv(r"D:\GENAI\01_PREREQUISTS\00_Python\08_Data_Visualization\Datasets\gayle-175.csv")
df
```

Styles & save

- Changing the style instantly upgrades the figure's look
- `plt.savefig()` — save as PNG/PDF; `dpi` controls quality
- Sessions confirm the theme = reporting quality also matters

PYTHON

```
plt.style.available
```

PYTHON

```
plt.style.use('dark_background')
```

Save figure

Once a figure looks right, you can export it: `plt.savefig('sample.png')` writes the current figure to a file on disk. A few practical rules for saving:

- `dpi` controls resolution — `dpi=300` gives crisp, print-quality output.
- `bbox_inches='tight'` trims the empty padding around the plot.
- Supported formats include PNG, PDF, SVG and JPG.
- Call `savefig()` **before** `plt.show()` — once `show()` runs, the figure is cleared.

In the code below we load a large NumPy array from a `.numpy` file, plot its histogram on a log scale (so that huge counts do not crush the small ones), and then save the finished picture as `sample.png` in the notebook folder.

PYTHON

```
arr = np.load(r"D:\GENAI\01_PREREQUISTS\00_Python\08_Data_Visualization\Datasets\big-array.npy")
plt.hist(arr, bins=[10, 20, 30, 40, 50, 60, 70], log=True)

plt.savefig('sample.png')
```

Summary

In this session you learned the fundamentals of Matplotlib, Python's classic plotting library. We started with `plt.plot()` for 2D line charts and made them readable with `plt.title()`, `plt.xlabel()`, `plt.ylabel()`, colors, line styles, markers, legends and grids. We then covered `plt.scatter()` for exploring relationships between two numerical columns, `plt.bar()` and `plt.barh()` for grouped and stacked category comparisons, `plt.hist()` for frequency distributions (including custom `bins` and a log-scaled y-axis), and `plt.pie()` for proportion-of-whole views with `autopct`, `explode` and `shadow`. Finally, we saw how `plt.style.use()` and `plt.savefig()` make figures publication-ready. The golden rule throughout: choose the chart based on the data type and the analysis — univariate or bivariate, numerical or categorical. You now have the toolkit to start every future analysis with a picture.

In this advanced session you added a whole new set of tools to your Matplotlib toolkit. Colored scatter plots encoded a third dimension through `c=` and `cmap`, with `plt.colorbar()` as the legend, and `plt.figure(figsize=...)` gave you control over canvas size. Annotations (`plt.text` and `plt.annotate`) plus reference lines (`plt.axhline`, `plt.axvline`) made scatter plots communicate insights directly. Subplots — both `plt.subplots(nrows, ncols)` and `fig.add_subplot(r, c, i)` — let you arrange many charts in one figure with axes-level control. We then moved into 3D: scatter and line plots via `projection='3d'`, surfaces built from `np.meshgrid` and a formula, and their 2D projections as `contour` and `contourf`. Finally, `plt.imshow()` handled matrix heatmaps and the Pandas `.plot(kind=...)` wrapper produced quick charts from DataFrames. You are now equipped to build rich, multi-dimensional, multi-panel figures for real analyses.

In this session you discovered Seaborn — Matplotlib's friendlier, prettier sibling. You learned the crucial distinction between axes-level functions (which draw on the current axes and are perfect for fine control) and figure-level functions like `relplot`, `displot` and `clustermap` (which own the figure and support faceting with `col` / `row`). You explored three plot families: relational plots (`scatterplot`, `lineplot`) with semantic mapping via `hue`, `style` and `size`; distribution plots (`histplot`, `kdeplot`, `rugplot`, and their bivariate forms) for univariate shape and outliers; and matrix plots (`heatmap` with `annot`, `clustermap` with dendrograms) for dense two-way data. The whole session ran on seaborn's built-in datasets — `tips`, `titanic`, `iris` — loaded from the internet with `sns.load_dataset`. You are now ready for categorical, regression and multi-panel plots, the territory of the next session.

This notebook completed your tour of Seaborn's categorical, regression and multi-panel tools. You learned the three families of categorical plots — scatter (`strippplot` with `jitter`, `swarmplot`), distribution (`boxplot` with its five-number summary, `violinplot` as box+KDE), and estimate (`barplot` / `pointplot` with error bars, `countplot` for pure counts) — all reachable through the single figure-level function `sns.catplot(kind=...)`. You saw how `hue`, `order`, `saturation`, `split` and `estimator` customize each kind, and how `col=` + `row=` turn them into facet grids. Regression entered the mix via `regplot` (axes-level, line + confidence band), `lmpplot` (figure-level with hue/facets) and `residplot` (model-fit diagnosis). Finally you mastered the grid objects: `FacetGrid` with `g.map` and `add_legend`, `PairGrid` with `map_diag` / `map_upper` / `map_lower` (and the one-line `pairplot`), and `JointGrid` with bivariate-plus-marginals (`jointplot`). Along the way, `sns.get_dataset_names()` and `sns.load_dataset()` unlocked Seaborn's built-in data. You can now go from raw tips to faceted, modelled, publication-ready multi-panel figures.

This notebook put style control at your fingertips. You saw that `sns.set_theme()` restores a consistent baseline, and that three families of settings shape every figure: **styles** (`sns.set_style` with `darkgrid`, `whitegrid`, `white`, `dark`, `ticks`) for the background, **contexts** (`sns.set_context` with `paper`, `notebook`, `talk`, `poster`) for proportional scaling of fonts and lines, and **palettes** (`sns.color_palette`, `sns.set_palette`) for coherent, accessible colour schemes across all plots in the notebook. Crucially, these settings are global and persistent — one `set_theme()` call near the top of a notebook explains why every chart downstream shares the same visual language. Pick a palette, pick a context, set it once, and your whole analysis looks like a designed product rather than a pile of defaults.

This session gave you the third major plotting framework: interactive charts via Plotly. You learned the roadmap — **Go** (low-level traces/layout/figure), **Express** (one-line `px` calls) and **Dash** (web apps) — and saw why Express is your default. You built the full Express catalogue on real data: scatters with `color`, `size`, `hover_name` and an animated `animation_frame`; line and bar charts (grouped, stacked, `text_auto` labels, animated continents); histograms with `nbins` and `color`; pie/donut, sunburst and treemap for hierarchical proportions; `imshow` heatmaps from pivots; 3D scatter via `px.scatter_3d` and `scatter_matrix` for pair slugs. You then stepped down to Go for what Express cannot do — `go.Surface` and `go.Contour` built from `np.meshgrid(z)` — and assembled households subplots with `make_subplots` + `add_trace(row, col)`. Every figure was interactive: hover, zoom, rotate, animate. Plotly is the natural choice for exploratory shared dashboards; combined with Seaborn for stats-first charts, your visualization toolkit is complete.

Advanced Matplotlib

Colored scatterplots

- `c=` — numeric category codes (colors)

- `cmap` — colormap ('jet', 'viridis', ...)
- `alpha` — transparency 0-1
- Categorical data → convert to numeric codes first

PYTHON

```
iris = pd.read_csv(r"D:\GENAI\01_PREREQUISITS\00_Python\08_Data_Visualization\Datasets\iris.csv")
iris.sample(5)
```

PYTHON

```
iris['Species'] = iris['Species'].replace({'Iris-setosa':0, 'Iris-versicolor':1, 'Iris-virginica':2})
iris.sample(5)
```

Deep Dive : Colored scatter — 3rd dimension via c

- `c` — color by a numeric column (categorical labels → numbers first)
- `cmap` — colormap ('viridis', 'plasma', 'RdYlGn', 'Set1')
- encode category in color — 3rd dimension on 2D plot
- `alpha` — transparency (overlap visualization)
- `colorbar`: `plt.colorbar()` legend for numeric scale
- mapping rule: min→first cmap color, max→last
- categorical dtype color mapping manually
- Use: species/class separation visually
- With hue real dots grouped clusters visible instantly
- Colored scatter = cheap clustering peek

PYTHON

```
iris['Species'] = iris['Species'].replace({'Iris-setosa':0, 'Iris-versicolor':1, 'Iris-virginica':2})
plt.scatter(iris['SepalLengthCm'], iris['PetalLengthCm'], c=iris['Species'], cmap='viridis')
```

Figure size

- `figsize=(width, height)` in inches
- Set the figure before rendering the plot
- Bigger figures — readability

PYTHON

```
plt.figure(figsize=(15, 7))
plt.scatter(...)
```

Deep Dive : Figure size & annotations

- `plt.figure(figsize=(w,h))` — set canvas inches BEFORE axes
- Larger figsize — readable dense scatter

- `text(x,y,label)` — free floating label at data point
- `annotate` — label + arrow to point (xy target, xytext label offset)
- `arrowprops` — arrow style config
- `annotate` outliers, extremes, insights
- Contextual labels — inform the analyst
- aspect policy: subplots param for equal axes
- guides for knee points / maxima
- callback patterns `sample_df` → `annotate` few

PYTHON

```
plt.figure(figsize=(15,7))

plt.scatter(iris['SepalLengthCm'],iris['PetalLengthCm'],c=iris['Species'],cmap='jet',alpha=0.7)
plt.xlabel('Sepal Length')
plt.ylabel('Petal Length')
plt.colorbar()
```

PYTHON

```
plt.figure(figsize=(15,7))      # wider canvas
plt.scatter(x, y)
plt.text(1, 5, 'Point 1')      # raw label at coords
plt.annotate('minimum', xy=(min_x,min_y),
            xytext=(10,10), arrowprops=dict(arrowstyle='->'))
```

Annotations

A scatter plot is often busy: hundreds of points and no way to tell who is who. Annotations solve that by writing text directly on the figure. `plt.text(x, y, label)` places free-floating text at the data coordinates `(x, y)` — perfect for labelling every point in a loop. `plt.annotate(...)` goes one step further: parameters like `xy`, `xytext` and `arrowprops` let you draw an arrow from a label position to a specific data point, so you can point at outliers, maxima or trends. In the example below we plot IPL batters with marker size proportional to runs (`s=sample_df['runs']`), then loop over the DataFrame adding a `plt.text()` label for every batter — turning a plain scatter into a self-explanatory analysis chart. The `fontdict` argument ('size', 'color') fine-tunes how each label looks.

- `text(x, y, label)` — label at data coordinates
- Highlight specific points (outliers)
- Write insights directly on the plot

PYTHON

```
batters = pd.read_csv(r"D:\GENAI\01_PREREQUISITS\00_Python\08_Data_Visualization\Datasets\batter.csv")
```

PYTHON

```
batters.shape
batters
```

Horizontal and Vertical lines

Reference lines are one of the fastest ways to put meaning on a plot. `plt.axhline(y)` draws a horizontal line that spans the full width of the axes at the given y value, and `plt.axvline(x)` draws the corresponding vertical line at a given x value. These are ideal for showing thresholds, targets or baselines — for example, drawing the average runs and average strike-rate of the league so that a heavily populated quadrant becomes obvious at once. You can pass `color` to style them ('red', 'green', 'black'), and multiple calls overlay several thresholds. Because the lines run edge-to-edge, they never need you to craft matching arrays. In our code the red lines mark the average values of the two axes, chopping the scatter into four insight-rich quadrants.

PYTHON

```
plt.figure(figsize=(18,10))
plt.scatter(sample_df['avg'],sample_df['strike_rate'],s=sample_df['runs'])

plt.axhline(130,color='red')
plt.axhline(140,color='green')
plt.axvline(30,color='red')

for i in range(sample_df.shape[0]):
    plt.text(sample_df['avg'].values[i],sample_df['strike_rate'].values[i],sample_df['batter'].values[i])
```

Horizontal / Vertical lines

- To show thresholds / baselines
- `axhline/axvline` — full-size line, `hlines/vlines` — segments

PYTHON

```
plt.axhline(y)    # horizontal reference line
plt.axvline(x)    # vertical reference line
```

Deep Dive : Horizontal/vertical lines

- `axhline` — infinite horizontal reference line (y)
- `axvline` — vertical corresponding
- Highlight thresholds — avg, target, cutoff
- cv lines full-width across axes
- dash style to not confuse with data
- Multiple thresholds — multiple calls
- legend labels → line meaning
- Used in anomaly flagging & decision boundaries
- Pair with annotation for reasoned labels

- easier than plotting short arrays as vertical segs

PYTHON

```
# A diff way to plot graphs
batters.head()
```

PYTHON

```
plt.figure(figsize=(15,6))
plt.scatter(batters['avg'],batters['strike_rate'])
plt.title('Something')
plt.xlabel('Avg')
plt.ylabel('Strike Rate')

plt.show()
```

Subplots

- `subplots(nrows, ncols)` — grid of axes
- `ax[row, col]` indexing (or `ax[0]` for 1-D)
- `sharex/sharey` — axis shared
- `fig.add_subplot(r,c,i)` — positional alternative

PYTHON

```
fig,ax = plt.subplots(figsize=(15,6))

ax.scatter(batters['avg'],batters['strike_rate'],color='red',marker='+')
ax.set_title('Something')
ax.set_xlabel('Avg')
ax.set_ylabel('Strike Rate')

fig.show()
```

PYTHON

```
fig, ax = plt.subplots(nrows=2, ncols=2, sharex=True, figsize=(10,10))
ax[0,0].scatter(...)
ax[0,1].plot(...)

fig = plt.figure()
ax1 = fig.add_subplot(2,2,1)
```

Deep Dive : Subplots — multiple figures one window

- `subplots(nrows, ncols)` → fig + array of Axes
- Plot on each ax explicitly: `ax[i].plot/scatter/set_xlabel`
- `sharex/sharey` — same scale across for compare
- indexing: 1D row-major (`ax[0]`, `ax[1]`); 2D (`ax[0,0]`...)

- `fig.add_subplot(r,c,index)` — manual incremental
- Titles per axis via `ax[i].set_title`
- Layout: aligned comparison & storytelling rows
- tweak: `fig.tight_layout()` spacing
- Multi-panel reports standard
- AXES-LEVEL control core of matplotlib

PYTHON

```
fig, ax = plt.subplots(nrows=2,ncols=1,sharex=True,figsize=(10,6))

ax[0].scatter(batters['avg'],batters['strike_rate'],color='red')
ax[1].scatter(batters['avg'],batters['runs'])

ax[0].set_title('Avg Vs Strike Rate')
ax[0].set_ylabel('Strike Rate')

ax[1].set_title('Avg Vs Runs')
ax[1].set_ylabel('Runs')
ax[1].set_xlabel('Avg')
```

PYTHON

```
fig, ax = plt.subplots(nrows=2,ncols=2,figsize=(10,10))

ax[0,0].bar(batters['batter'],batters['avg'])
ax[0,1].scatter(batters['avg'],batters['runs'])
ax[1,0].hist(batters['avg'])
ax[1,1].hist(batters['runs'])
```

3D plots

- `projection='3d'` — 3D axes
- scatter / line / surface all possible
- 3 cols data → 3D space

PYTHON

```
ax = plt.subplot(projection='3d')
ax.scatter(x, y, z)
```

Deep Dive : 3D plots & surfaces

- `projection='3d'` — new axes, third dim
- `meshgrid` — all (x,y) coordinate pairs grid
- `plot_surface` — height map surface
- `contour`, `contourf` — level curves top-down

- 3D reveals 3-variable relationships
- Contour = 2D projection of height
- functions of two variables visualized
- Rotatable interactive (notebook)
- depth perception — helpful overview
- Extra dims beyond 3 → animation/color code

PYTHON

```
batters

fig = plt.figure()

ax = plt.subplot(projection='3d')

ax.scatter3D(batters['runs'],batters['avg'],batters['strike_rate'],marker='+')
ax.set_title('IPL batsman analysis')

ax.set_xlabel('Runs')
ax.set_ylabel('Avg')
ax.set_zlabel('SR')
```

PYTHON

```
fig = plt.figure()
ax = plt.subplot(projection='3d')
ax.scatter(x, y, z)          # 3D scatter
ax.plot(x, y, z)             # 3D line

xx, yy = np.meshgrid(x, y)  # 2D grids for surface
z = xx**2 + yy**2
ax.plot_surface(xx, yy, z, cmap='viridis') # surface
ax.contour(xx, yy, z)        # contour lines
```

3D Line Plot

A 3D line plot traces a curve that moves through all three dimensions. You create a 3D axis with `plt.subplot(projection='3d')` and then draw with `ax.plot3D(x, y, z, color='red')` — the curve passes through the ordered points `(x, y, z)`. It is often useful to mark the exact vertices with `ax.scatter3D(x, y, z, s=[100, 100, 100, 100])`, whose `s` parameter sets the marker size. In our example the points `(0,0,0)`, `(1,10,13)`, `(5,13,20)` and `(25,0,9)` form a closed loop that rises and falls — a small shape float you can rotate interactively in the notebook. 3D line plots are great for paths over time (a satellite orbit, a moving price in price/volume/time space) whenever a third coordinate matters.

PYTHON

```
x = [0,1,5,25]
y = [0,10,13,0]
z = [0,13,20,9]

fig = plt.figure()

ax = plt.subplot(projection='3d')

ax.scatter3D(x,y,z,s=[100,100,100,100])
ax.plot3D(x,y,z,color='red')
```

3D Surface Plots

A surface plot is the natural 3D extension of a line plot: instead of a curve, we draw a warped sheet — the height of every (x, y) point given by a function $z = f(x, y)$. The recipe is always the same: create 1D x and y vectors with `np.linspace`, then turn them into 2D grids with `np.meshgrid`, apply the formula element-wise to build z , and finally call `ax.plot_surface(xx, yy, z, cmap='viridis')` on a 3D axis. `meshgrid` produces every combination of x and y coordinates, so z becomes a full grid and the surface is smooth. Matplotlib lets us experiment freely with formulas: $z = xx^2 + yy^2$ is a paraboloid, $z = np.sin(xx) + np.cos(yy)$ wavelike, and `fig.colorbar(p)` adds a legend for the height.

PYTHON

```
x = np.linspace(-10,10,100)
y = np.linspace(-10,10,100)
```

3D surface / contour

- `meshgrid` — 2D coordinate grid
- `plot_surface` — 3D surface
- `contour` — lines; `contourf` — filled colors
- $Z = \text{function}(x,y)$ — for experimenting with formulas

PYTHON

```
xx, yy = np.meshgrid(x,y)
```

PYTHON

```
z = xx**2 + yy**2
z.shape
```

Contour Plots

A contour plot takes the same $z = f(x, y)$ grid used for a 3D surface and projects it down onto the 2D plane, drawing level curves — lines of constant height, like the rings on a topographic map. `ax.contour(xx, yy, z, cmap='viridis')` draws thin lines for each level, while `ax.contourf(xx, yy, z, cmap='viridis')` fills the areas between levels with color, making the shape easier to read. Each color band represents a range of

`z` values, and `fig.colorbar(p)` maps colors back to numbers. Contours reveal the *layout* of a function instantly: `z = sin(xx) + cos(yy)` appears as a pattern of alternating ridges (peaks) and valleys (troughs). They are indispensable for understanding functions of two variables before you ever fit a model to them.

PYTHON

```
fig = plt.figure(figsize=(12,8))

ax = plt.subplot()

p = ax.contour(xx,yy,z,cmap='viridis')
fig.colorbar(p)
```

PYTHON

```
fig = plt.figure(figsize=(12,8))

ax = plt.subplot()

p = ax.contourf(xx,yy,z,cmap='viridis')
fig.colorbar(p)
```

Heatmap (imshow)

A heatmap renders a 2D matrix as a grid of colored cells, where color encodes the value at each cell.

`plt.imshow(grid)` takes a matrix-shaped input and paints each entry; `plt.colorbar()` then tells you which color means which number. Because the input must already be a matrix, you usually build one first with a Pandas `pivot_table(index=..., columns=..., values=..., aggfunc=...)`. The axis labels of an image are integer indices, so `plt.yticks(...)` and `plt.xticks(...)` are used to restore meaningful names — here overs (1-20) on the rows and ball numbers (1-6) on the columns. Heatmaps are perfect for looking at dense two-way patterns: run-scoring tendencies by over, correlation matrices, or any table where row-by-row scanning would be hopeless.

Heatmap

Functional tools

map/filter apply functions; reduce aggregates. Often paired with lambdas.

PYTHON

```
delivery = pd.read_csv(r"D:\GENAI\01_PREREQUISITS\00_Python\08_Data_Visualization\Datasets\IPL_Ball_by_Ball")
delivery.head()
```

Heatmaps (imshow)

- `imshow` — 2D matrix → color-encoded grid
- `grid` needs to be a `pivot_table` / in matrix shape
- `colorbar` — value scale
- Set the ticks for categorical axis labels

PYTHON

```
plt.imshow(grid)
plt.colorbar()
plt.yticks(...); plt.xticks(...)
```

Deep Dive : Heatmap — imshow matrix

- `imshow(matrix)` — 2D grid as color cells
- `pivot_table` → matrix (rows×cols) first
- color = value magnitude immediate
- `yticks/xticks` — map row/col indexes to labels
- `colorbar` — legend scale
- `annotate` numeric (seaborn.heatmap `annot=True` better)
- Use: run-scoring patterns, correlation matrices
- Dark = low, bright = high (or reverse `cmap`)
- density overview compact — heat map thousands values
- seaborn heatmap has `annot` + nicer defaults

PYTHON

```
temp_df = delivery[(delivery['ballnumber'].isin([1,2,3,4,5,6])) & (delivery['batsman_run']==6)]
```

PYTHON

```
grid = temp_df.pivot_table(index='overs', columns='ballnumber', values='batsman_run', aggfunc='count')
```

Pandas Plot()**Pandas fundamentals**

Labeled axes, alignment, indexing, grouping, missing data, I/O. Critical for EDA/preprocessing.

AI/ML relevance

Shapes, dtypes, vectorization, indexing, and clean tabular ops are critical for reliable data pipelines.

- DataFrame/Series `.plot()` — a matplotlib wrapper
- `kind` — pie/bar/line/hist/scatter...
- On a dataframe the index is the default x-axis
- `subplots=True` — a separate plot for each column
- Stacked bars, grouped plots — readable analysis

PYTHON

```
s.plot(kind='pie')
df.plot(kind='scatter', x='col1', y='col2')
df.plot(kind='bar', stacked=True)
df.plot(kind='line', x='Date')
df[['m1', 'm2']].plot(kind='hist', bins=40)
df[['m1', 'm2', 'm3']].plot(kind='pie', subplots=True)
```

Deep Dive : pandas plot() — quick charts

- `.plot(kind=...)` — DataFrame/Series native quick chart
- kinds: line, bar, barh, pie, hist, box, scatter, hexbin, kde, area
- `subplots=True` — each col separate panel
- legend, figsize, stacked params pass-through
- Time x values — line plots fine
- dataset m-mehdi pandas tutorial online (dataset var)
- wide/multiindex pivots also plot-able
- fast EDA defaults over control fidelity
- choose when prototyping visuals vs custom matplotlib
- input dims: numeric cols

PYTHON

```
# on a series

s = pd.Series([1,2,3,4,5,6,7])
s.plot(kind='pie')
```

PYTHON

```
import seaborn as sns
tips = sns.load_dataset('tips')
```

Plotting with Seaborn

Seaborn Roadmap

Types of Functions

- Figure Level
- Axis Level

Main Classification

- Relational Plot
- Distribution Plot

- Categorical Plot
- Regression Plot
- Matrix Plot
- Multiplots

<https://seaborn.pydata.org/api.html>

1. Relational Plots

Relational plots answer "how do two variables move together?". They are the heart of **bivariate analysis**. When one of the variables is categorical or you simply want to see every observation, use a **scatterplot**; when the x-axis is continuous (like a `year`) and you want the *trend*, use a **lineplot**. Both are available as axes-level functions (`sns.scatterplot`, `sns.lineplot`) and through the figure-level `sns.relplot(kind='scatter' | 'line', ...)`. The magic is in the semantics:

- `hue='sex'` — color points by a category.
- `style='time'` — change marker shape per category.
- `size='size'` — scale point size by a numeric column.

Because the figure-level `relplot` returns a `FacetGrid`, it also accepts `col` and `row` to split the data into a grid of panels automatically — something the axes-level functions cannot do. Comparison across countries, across time, across groups — relational plots are your first stop for almost any pairing of numeric columns.

PYTHON

```
import seaborn as sns
import matplotlib.pyplot as plt
import plotly.express as px
```

PYTHON

```
tips = sns.load_dataset('tips')
tips
```

Deep Dive : Seaborn — figure-level vs axes-level

- sidebar style: no seaborn minimal, high-level — layer atop matplotlib
- axes-level (scatterplot, lineplot, histplot...) — reuse matplotlib figure/axes grid
- figure-level (relplot/displot/...) — auto-creates multi-panel fig
- `hue=col` — color encode variable; `style=` col dashes/markers; `size=` col point sizes
- Semantic mapping → 4 variables one scatter
- figure-level supports col/row faceting; axes-level not
- seaborn handles stats (KDE smoothing, summary) out of box
- choose: quick expressive (figure-level) vs precise control (axes-level)

PYTHON

```
# scatter plot -> axes level function
sns.scatterplot(data=tips, x='total_bill', y='tip', hue='sex', style='time', size='size')
```

PYTHON

```
# relplot -> figure level -> square shape
sns.relplot(data=tips, x='total_bill', y='tip', kind='scatter', hue='sex', style='time', size='size')
```

Deep Dive : Line plots + faceting

- lineplot — trend over continuous x (year)
- hue — separate lines in color; style — dash patterns
- relplot(kind='line'|='scatter') — figure level — auto legend+
- faceting: col= — separate panel per category value
- col_wrap — multiple columns wrap rows
- row= — second facet dimension
- Gapminder India/Brazil trend — multi-line compare
- Largest: facet plot = one shortcut for many subplots
- semantics map col vars → visual channels — readability
- .inunching & random samples explore grids

PYTHON

```
# Line plot
gap = px.data.gapminder()
temp_df = gap[gap['country'] == 'India']
temp_df
```

PYTHON

```
# axes level function
sns.lineplot(data=temp_df, x='year', y='lifeExp')
```

2. Distribution Plots

Distribution plots are built for **univariate analysis** — studying one column at a time. Their questions: What is the *range* of values? What is the *central tendency*? Is the data *bimodal* (two peaks)? Are there *outliers*? Three tools cover most needs:

- `sns.histplot` — the familiar histogram: data is binned into intervals and each bin is drawn as a bar whose height is the count (the frequency).
- `sns.kdeplot` — kernel density estimate: instead of hard bins, each observation smears into a smooth Gaussian kernel and the kernels add up into a continuous density curve.

- `sns.rugplot` — little ticks along the axis marking every single observation; a lightweight complement drawn beneath any other plot.

The figure-level `sns.displot(kind='hist' | 'kde' | 'ecdf')` wraps all of them and adds `hue`, `fill`, and faceting. A histogram can even be applied to a categorical variable as a count of rows per category. This section is about shape: the distribution tells you whether data clusters around a mean, stretches right, or splits into peaks.

Deep Dive : Distribution plots — hist/kde/rug

- Distribution plots — univariate analysis (one col)
- figure-level displot wraps hist/kde/rug/ecdf
- bins — bucket count/edges
- hue — separate distributions per class
- element='step' — outline (overlap readable)
- kde — smooth continuous density (sum of kernels)
- fill, bw_adjust, cut params
- rugplot — every observation tick (all raw positions)
- bivariate (x+y) — 2D hist heat & kde contour
- Compare DAILY numeric spread, age by sex
- Categorical dist can use x=cat with hue num

PYTHON

```
# plotting univariate histogram
sns.histplot(data=tips, x='total_bill')
```

PYTHON

```
sns.displot(data=tips, x='total_bill', kind='hist')
```

2. Matrix Plots

Matrix plots visualize a rectangular grid where each cell is colored by its numeric value — the fastest way to see a two-way pattern in a dense table.

- `sns.heatmap(matrix)` renders a 2D array as colored cells: index on one axis, columns on the other, and color encoding magnitude. `annot=True` writes the exact numbers inside the cells, `linewidths` separates cells with thin lines, and `cmap` picks the palette. You usually build the matrix first with a Pandas `pivot(index=..., columns=..., values=...)` or `pivot_table(...)`.
- `sns.clustermap(matrix)` is a heatmap plus **hierarchical clustering**: seaborn groups similar rows and columns together and draws dendrograms along the edges, then reorders the matrix so related items sit adjacent. Clusters pop out as blocks of similar color.

Heatmaps are the go-to for correlation matrices and pivot summaries; clustermaps reveal hidden groups in the same data. When a grid has thousands of numbers, stop reading cells and let the colors talk.

Deep Dive : Matrices — heatmap & clustermap

- heatmap — 2D matrix color-encoded (annot adds values)
- pivot first — long→wide rows×cols matrix
- linewidth — cell spacing; cmap palette
- best: correlation matrices, pivot summaries
- clustermap — heatmap + hierarchical clustering dendrograms
- reorders rows/cols to reveal patterns/groups
- rows similar → clusters block diagonal visual
- Useful: find related variables/regions
- annot large matrices overwhelming — subset
- seaborn = publication-ready matrix visuals

PYTHON

```
# Heatmap

# Plot rectangular data as a color-encoded matrix
temp_df = gap.pivot(index='country',columns='year',values='lifeExp')

# axes level function
plt.figure(figsize=(15,15))
sns.heatmap(temp_df)
```

PYTHON

```
# annot
temp_df = gap[gap['continent'] == 'Europe'].pivot(index='country',columns='year',values='lifeExp')

plt.figure(figsize=(15,15))
sns.heatmap(temp_df,annot=True,linewidth=0.5, cmap='summer')
```

Seaborn Part 2 — Categorical, Regression & Multi-Panel Grids

Categorical Plots

Categorical Scatter Plot

- Stripplot
- Swarmplot

Categorical Distribution Plots

- Boxplot
- Violinplot

Categorical Estimate Plot -> for central tendency

- Barplot
- Pointplot
- Countplot

Figure level function -> `catplot`

Recap — the relational scatter, now with tips

Cell 3 opens your categorical session with a familiar sight: `sns.scatterplot(data=tips, x='total_bill', y='tip')`. This is the exact relational plot from Session 25 — one point per dinner bill, total amount spent on the x-axis against the tip on the y-axis. Hold this image in your head. As you scroll through the notebook, notice how the very same `tips` data gets progressively richer treatments: first the raw scatter, then the categorical scatter that compresses the x-axis into a few categories, then boxplots that summarize those categories, and finally regression lines that model the trend. Each new plot is one more step up the ladder from *raw points* → *grouped points* → *distribution summaries* → *statistical estimates* → *models*. Watch for the moment when the scatter of all the points becomes a swarm of dots per eating time-of-day (`day`): that transition is the heart of this notebook.

PYTHON

```
sns.scatterplot(data=tips, x='total_bill', y='tip')
```

Deep Dive : Regression plots — regplot/lmplot/residplot

- regplot — scatter + linear regression line + CI band
- lmplot — figure-level regression with hue/facets
- residplot — fit at residuals vs x (model fit check)
- Visual model relationship strength
- hue in lmplot not available in regplot
- outliers influence regression line visibly
- ci — confidence interval shading
- order= / logistic variants (nonlinear)
- pair plots with reg kind extensions
- Quick hypothesis visual validation

PYTHON

```
sns.regplot(data=tips, x='total_bill', y='tip')      # scatter + fitted line
sns.lmplot(data=tips, x='total_bill', y='tip', hue='sex')  # facets
sns.residplot(data=tips, x='total_bill', y='tip')      # residuals
```

Deep Dive : Categorical plots — strip/swarm

- catplot — FIGURE-level for categorical x
- kinds: strip, swarm, box, violin, boxen, point, bar, count
- strip — every point raw; jitter offsets overlaps

- swarm — points avoid overlap (beeswarm)
- hue — split color per category inside group
- one categorical + one numerical → distribution comparison per category
- overlay many obs → see exact spread & clusters
- vs scatterplot — categorical axis compressed
- swarm best clarity <200 pts
- axes-level variants: `sns.stripplot` / `sns.swarmplot`

PYTHON

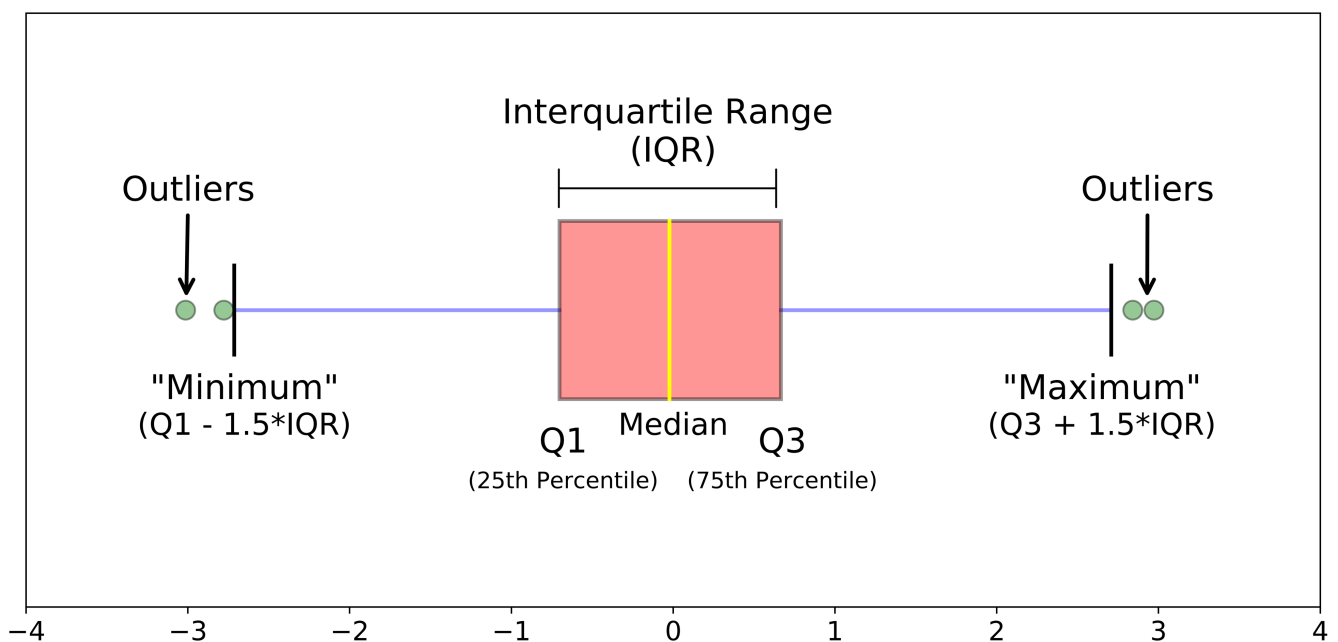
```
# strip plot
# axes level function
sns.stripplot(data=tips, x='day', y='total_bill')
```

PYTHON

```
# using catplot
# figure level function
sns.catplot(data=tips, x='day', y='total_bill', kind='strip')
```

Boxplot

A boxplot is a standardized way of displaying the distribution of data based on a five number summary ("minimum", first quartile [Q1], median, third quartile [Q3] and "maximum"). It can tell you about your outliers and what their values are. Boxplots can also tell you if your data is symmetrical, how tightly your data is grouped and if and how your data is skewed.



Deep Dive : Box plots — 5-number summary

- Box: min, Q1, median, Q3, max (whiskers = $1.5 \cdot IQR$)

- Points beyond whiskers = OUTLIERS
- Box width = IQR; line inside = median
- `catplot kind='box'` — figure-level facets
- `hue` — separate box per subclass (`day×sex`)
- Single numeric — one box overall
- $IQR = Q3 - Q1$; whisker bounds standard
- Compare medians across categories
- Better than bar: full distribution summary
- Data cleaning check: outliers visible

PYTHON

```
# Box plot
sns.boxplot(data=tips,x='day',y='total_bill')
```

PYTHON

```
# Using catplot
sns.catplot(data=tips,x='day',y='total_bill',kind='box')
```

Deep Dive : Violin — box + density

- Violin = boxplot + KDE — distribution shape + stats
- Width of violin = density of points
- Inside: mini boxplot (median, IQR)
- One distribution per category
- `split=True` — hue two sides same violin
- Shows multimodal shapes boxes miss
- Check skew/peaks/bimodal
- Better insight than plain box
- `ci/bootstrap` irrelevant
- cautious subset: `catplot` facet combination

PYTHON

```
# violinplot
sns.violinplot(data=tips,x='day',y='total_bill')
```

PYTHON

```
sns.catplot(data=tips,x='day',y='total_bill',kind='violin')
```

Deep Dive : barplot / pointplot / countplot

- `barplot` — mean of y per category + error bar

- pointplot — mean points + confidence lines (trend across cats)
- countplot — COUNT observations (no y needed) — histogram categorical
- ci/errorbar: None disables confidence interval
- stat='median'/'sum' alternatives
- hue — subclassed bars
- Categorical summary — where shall values per group
- bar vs histogram vs count: bar=agg value, count=number rows
- Use distribution first, summary second
- Statistical aggregates encode messages

When there are multiple observations in each category, it also uses bootstrapping to compute a confidence interval around the estimate, which is plotted using error bars

A special case for the bar plot is when you want to show the number of observations in each category rather than computing a statistic for a second variable. This is similar to a histogram over a categorical, rather than quantitative, variable

PYTHON

```
# barplot
# some issue with errorbar
import numpy as np
sns.barplot(data=tips, x='sex', y='total_bill', hue='smoker', estimator=np.min)
```

PYTHON

```
sns.barplot(data=tips, x='sex', y='total_bill', ci=None)
```

Regression Plots

- regplot
- lmpplot

In the simplest invocation, both functions draw a scatterplot of two variables, x and y, and then fit the regression model $y \sim x$ and plot the resulting regression line and a 95% confidence interval for that regression.

Regression plots — regplot, lmpplot, residplot

A natural step after describing data is to *model* it. `sns.regplot(data, x='total_bill', y='tip')` draws the scatter and then fits a simple linear regression `tip ~ total_bill`, plotting the fitted line plus a shaded 95% confidence band for the mean. `regplot` is an **axes-level** function — it draws on the current axes and therefore has no `hue` parameter. `sns.lmpplot(...)` is its **figure-level** sibling: it builds a `FacetGrid` internally, so it accepts `hue`, `col` and `row`, and is the right choice when you want separate regression lines per group (here, per `sex`). `sns.residplot(data, x='total_bill', y='tip')` plots the **residuals** on the y-axis — every point's vertical distance from the regression line. If the residuals form a random cloud around zero, a

straight line is a good fit; if they curve or fan out, linearity or constant variance is violated. These three plots are your first look at inference: not just *what* the data looks like, but *how strongly* and *how well* two variables relate.

PYTHON

```
# axes Level
# hue parameter is not available
sns.regplot(data=tips,x='total_bill',y='tip')
```

PYTHON

```
sns.lmplot(data=tips,x='total_bill',y='tip',hue='sex')
```

Faceting with catplot — col and row

So far every `catplot(kind=...)` produced a single panel. The moment you add `col='smoker'` and `row='time'`, the same call multiplies itself into a **grid of panels** — one for every combination of the two faceting columns (here $2 \times 2 = 4$ panels). Faceting is more powerful than a `hue` mapping: rather than splitting colors inside one crowded axes, each category gets its *own* clean panel, so the shapes of all the boxplots remain legible. You can combine `kind`, `hue`, `col` and `row` freely — e.g. a violin per `day` (cols) for each `time` (rows), colored by `sex`. Facets answer the truly multi-variate question: *does the pattern hold for both smokers and non-smokers, at lunch and at dinner?* Whenever a figure-level function feels cramped, reach for `col=` and `row=` before you give up on readability.

PYTHON

```
# figure level -> relplot -> displot -> catplot -> lmplot
sns.catplot(data=tips,x='sex',y='total_bill',kind='violin',col='day',row='time')
```

Deep Dive : FacetGrid / PairGrid / JointGrid

- FacetGrid — manual col×row×hue facet control + `g.map(fn,...)`
- PairGrid — every numeric×numeric pair matrix (diag=hist)
- pairplot — convenient default version
- JointGrid — joint distribution 2D + marginals (jointplot default)
- `g.map` computes plotting per facet
- utility: multivariate pattern inspection in one window
- color/semantics channel use groups
- export to fig via `.fig`

PYTHON

```
g = sns.FacetGrid(data=tips,col='day',row='time',hue='smoker')
g.map(sns.boxplot,'sex','total_bill')
g.add_legend()
```

PYTHON

```
g = sns.FacetGrid(data=tips, col='day', row='time', hue='smoker')
g.map(sns.boxplot, 'total_bill', 'tip')           # custom per-facet plot

sns.pairplot(iris, hue='species')                # all numeric pairs
g = sns.PairGrid(data=iris, hue='species')
g.map(sns.scatterplot); g.map_diag(sns.histplot)  # upper/lower choices

sns.jointplot(data=tips, x='total_bill', y='tip', kind='hist', hue='sex')
g = sns.JointGrid(data=tips, x='total_bill', y='tip')
g.plot(sns.kdeplot, sns.violinplot)
```

Plotting Pairwise Relationship (PairGrid Vs Pairplot)

PYTHON

```
sns.pairplot(iris, hue='species')
```

PYTHON

```
# pair grid
g = sns.PairGrid(data=iris, hue='species')
# g.map
g.map(sns.scatterplot)
```

JointGrid Vs Jointplot

PYTHON

```
sns.jointplot(data=tips, x='total_bill', y='tip', kind='hist', hue='sex')
```

PYTHON

```
g = sns.JointGrid(data=tips, x='total_bill', y='tip')
g.plot(sns.kdeplot, sns.violinplot)
```

Deep Dive : Built-in datasets & utility

- seaborn bundles sample datasets (tips, iris, titanic, planets, penguins...)
- load_dataset — read into DataFrame (from remote)
- get_dataset_names — view inventory
- Perfect quick prototyping without files
- offline fallback — pd.read_csv('...local...')
- machine learning demo datasets standard
- exercise data used across this course

PYTHON

```
# get dataset names
sns.get_dataset_names()
```

PYTHON

```
# Load dataset
sns.load_dataset('planets')
```

Seaborn Part 2 — Themes, Styles, Contexts & Colour Palettes

Themeing

- `set_theme`

Set aspects of the visual theme for all matplotlib and seaborn plots.

- `axes_style`

Get the parameters that control the general style of the plots.

- `set_style`

Set the parameters that control the general style of the plots.

- `plotting_context`

Get the parameters that control the scaling of plot elements.

- `set_context`

Set the parameters that control the scaling of plot elements.

- `set_color_codes`

Change how matplotlib color shorthands are interpreted.

- `reset_defaults`

Restore all RC params to default settings.

- `reset_orig`

Restore all RC params to original settings (respects custom rc).

`set_theme` function :

This function is used to set the theme of your plots, it can take a variety of inputs such as 'darkgrid', 'whitegrid', 'dark', 'white' or 'ticks'.

Example:

PYTHON

```
tips.head()
```

PYTHON

```
sns.barplot(x=["A", "B", "C"], y=[1, 3, 2])
```

Deep Dive : `set_theme` — global plot style

- `set_theme` — global defaults: style + palette + context + font
- style values: 'white', 'dark', 'whitegrid', 'darkgrid', 'ticks'
- grid → readability; white ⇒ clean print
- Applies to all subsequent seaborn/matplotlib plots
- Runs once at notebook start for consistency
- `.set()` — persistent | `axes_style(context manager)` — temporary
- Example bars show instantly visual difference
- matplotlib affects too; default keys in seaborn style dict
- choose aesthetic — background choice determines tint
- call early — later plots consistent

PYTHON

```
# Using whitegrid
sns.set_theme(style='whitegrid')
sns.barplot(x=["A", "B", "C"], y=[1, 3, 2])
```

PYTHON

```
# Using dark background
sns.set_theme(style='dark')
sns.barplot(x=["A", "B", "C"], y=[1, 3, 2])
```

axes_style function :

This function is used to set the style of the axes of your plots. It can take a variety of inputs such as 'white', 'dark', 'ticks' or a dictionary with key-value pairs of valid style options.

PYTHON

```
# Example:
sns.axes_style(style = 'white')
sns.barplot(x=["A", "B", "C"], y=[1, 3, 2])
```

Deep Dive : axesstyle & setcontext

- axes_style — coordinated axes look (grid, ticks, spines)
- context manager — style applies INSIDE `with` block only → scoped
- set_context: 'paper', 'notebook', 'talk', 'poster' — scale text/lines
- rcParams interplay: fonts, gridsize, tick sizes
- palette/font scale passing
- per-block style uniqueness — presentation vs notebook
- visual size ladder small→huge output
- Multiple axes_style: dict override
- Readable slide figures → context 'talk'/'poster'
- grid width/font relation matches labelling

SQL

```
# Use the function as a context manager to temporarily change the style of your plots:

with sns.axes_style("white"):
    sns.barplot(x=[1, 2, 3], y=[2, 5, 3])
```

SQL

```
# Load the iris data
iris = sns.load_dataset("iris")

# Set the style of the axes to "darkgrid"
sns.set_style("whitegrid")

# Create a scatter plot of sepal length vs sepal width
sns.scatterplot(x="sepal_length", y="sepal_width", data=iris)

# Show the plot
plt.show()
```

Scaling Figure Styles - `sns.set_context()`

Matplotlib allows you to generate powerful plots, but styling those plots for different presentation purposes is difficult. Seaborn makes it easy to produce the same plots in a variety of different visual formats so you can customize the presentation of your data for the appropriate context, whether it be a research paper or a conference poster.

You can set the visual format, or context, using `sns.set_context()`

Within the usage of `sns.set_context()`, there are three levels of complexity:

- Pass in one parameter that adjusts the scale of the plot
- Pass in two parameters - one for the scale and the other for the font size
- Pass in three parameters - including the previous two, as well as the rc with the style parameter that you want to override

Scaling Plots

Seaborn has four presets which set the size of the plot and allow you to customize your figure depending on how it will be presented.

In order of relative size they are: `paper`, `notebook`, `talk`, and `poster`. The notebook style is the default.

PYTHON

```
sns.set_style("ticks")

# Smallest context: paper
sns.set_context("paper")
sns.stripplot(x="day", y="total_bill", data=tips)
```

PYTHON

```
sns.set_style("ticks")

# Largest Context: poster
sns.set_context("poster")
sns.stripplot(x="day", y="total_bill", data=tips)
```

Scaling Fonts and Line Widths

You are also able to change the size of the text using the `font_scale` parameter for `sns.set_context()`

You may want to also change the line width so it matches. We do this with the `rc` parameter, which we'll explain in detail below.

While you're able to change these parameters, you should keep in mind that it's not always useful to make certain changes. Notice in this example that we've changed the line width, but because of its relative size to the plot, it distracts from the actual plotted data.

PYTHON

```
# Set font scale and reduce grid line width to match
sns.set_style("darkgrid")

sns.set_context("poster", font_scale = .5, rc={"grid.linewidth": 1.5})
sns.stripplot(x="day", y="total_bill", data=tips)
```

PYTHON

```
# Set font scale and increase grid line width to match
sns.set_context("poster", font_scale = .8, rc={"grid.linewidth": 5})
sns.stripplot(x="day", y="total_bill", data=tips)
```

The RC Parameter

As we mentioned above, if you want to override any of these standards, you can use `sns.set_context` and pass in the parameter `rc` to target and reset the value of an individual parameter in a dictionary. `rc` stands for the phrase 'run command' - essentially, configurations which will execute when you run your code.

PYTHON

```
# Plotting context function

sns.plotting_context()
# These are the property you can tweak in rc parameter
```

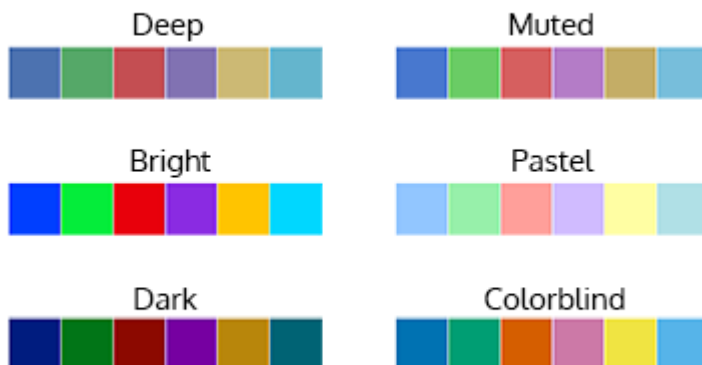
`seaborn.set_color_codes(palette='deep')`

Change how matplotlib color shorthands are interpreted.

Calling this will change how shorthand codes like "b" or "g" are interpreted by matplotlib in subsequent plots.

Parameters:

`palette` : {deep, muted, pastel, dark, bright, colorblind} Named seaborn palette to use as the source of colors.

**Deep Dive : setcolorcodes & palettes**

- color_codes: matplotlib letter shortcuts ('b','g','r','c','m','y','k')
- palette swap — RIDE letter → different shade
- deep/pastel — distinct family volumes
- convenient for existing matplotlib-style code
- set_palette — global color cycle for seaborn plots
- color_palette('deep', n) — generate n colors
- custom palette: hex list → color_palette([...])
- as_cmap=True — palette→heatmap colormap
- palettes: qualitative (deep/pastel/husl), sequential (viridis/magma), diverging
- ColorBrand coherent visuals

PYTHON

```
# 'b' color code with deep palette
sns.set_color_codes(palette='deep')
sns.relplot(data=tips, x='total_bill', y='tip', color='b')
```

PYTHON

```
# 'b' color code with pastel palette
sns.set_color_codes(palette='pastel')
sns.relplot(data=tips, x='total_bill', y='tip', color='b')
```

Color palettes

- `set_palette`

Set the matplotlib color cycle using a seaborn palette.

- `color_palette`

Return a list of colors or continuous colormap defining a palette.

- `husl_palette`

Return hues with constant lightness and saturation in the HUSL system.

- `hls_palette`

Return hues with constant lightness and saturation in the HLS system.

- `cubehelix_palette`

Make a sequential palette from the cubehelix system.

- `dark_palette`

Make a sequential palette that blends from dark to color.

- `light_palette`

Make a sequential palette that blends from light to color.

- `diverging_palette`

Make a diverging palette between two HUSL colors.

- `blend_palette`

Make a palette that blends between a list of colors.

- `xkcd_palette`

Make a palette with color names from the xkcd color survey.

- `crayon_palette`

Make a palette with color names from Crayola crayons.

- `mpl_palette`

Return a palette or colormap from the matplotlib registry.

`color_palette`

<https://seaborn.pydata.org/generated/seaborn.colorpalette.html#seaborn.colorpalette>

In Seaborn, the `color_palette()` function allows you to easily specify the colors for your plots. You can use pre-defined palettes, such as "deep", "muted", "pastel", "bright", "dark", and "colorblind", or you can create your own custom palette.

When using a pre-defined palette, you can specify the number of colors you want to use by passing in the desired number as the argument.

For example, using the "deep" palette and specifying 6 colors will return an array of 6 RGB color codes that can be used in your plot.

Deep Dive : `color_palette` & `cubehelix`

- `color_palette(name, n)` — return list of n RGB tuples
- named: deep/muted/bright/pastel/dark, hls/husl, Sequential, diverging
- custom list of hex → explicit color wants
- `set_palette` — global default color for future plots (pythonic)
- `cubehelix`: perceptually uniform rainbow sequence — great heatmaps
- start/rot/gamma control shift; light/dark bounds; reverse flip
- `barplot palette`= same list reuse
- color choice affects categorical distinction
- sequential for ordered data value; qualitative for labels
- nice formatting: `sns.color_palette("Paired")` — paired groups

You can also create your own custom color palette by passing in a list of RGB color codes.

The `as_cmap` parameter in seaborn's `color_palette` function is a boolean flag that, when set to True, returns a colormap object instead of a list of RGB values. This can be useful when plotting data that needs to be colored based on a continuous variable, such as a heatmap or a 2D histogram. The colormap can then be passed to other plotting functions, such as `heatmap` or `imshow`, to color the plotted data. An example of using *colorpalette with as_cmap* is:

PYTHON

```
deep_colors = sns.color_palette("deep", 6)

# Just to show the color palette using palplot
sns.palplot(deep_colors)
plt.show()
```

PYTHON

```
colors = ["#00CD00", "#00FFAA", "#FCCC00", "#FF0000"]
colors = sns.color_palette(colors)
sns.palplot(colors)
plt.show()
```

Deep Dive : Color to colormap (cmap)

- `colorpalette(...).as_cmap` — palette of discrete colors → continuous colormap
- sequential palettes ('Blues','Greens','Reds','viridis','rocket') → heatmaps
- custom hex list → 2/3-stop colormap
- `corr()` → correlation matrix cell colors
- readability: choose sequential for magnitude; diverging for +/- signed
- palette param passed to plotting (heatmap `cmap=` overrides)
- `hexcolors = sns.colorpalette("husl", 8, 0.7).as_hex()`
- consistent color language across graphs
- matrix heatmaps practitioners always use `cmap`
- `as_cmap` comes into play with `sns heatmap` directly

PYTHON

```
data = tips.corr()

# Create a colormap from a seaborn color palette
cmap = sns.color_palette("Blues", as_cmap=True)

# Create a heatmap using the colormap
sns.heatmap(data, cmap=cmap)
plt.show()
```

PYTHON

```
data = tips.corr()

# Create a colormap using your own custom color code
colors = ["#FFFF00", "#00FFAA", "#FCCDD", "#FCCC00"]
cmap = sns.color_palette(colors, as_cmap=True)

# Create a heatmap using the colormap
sns.heatmap(data, cmap=cmap)
plt.show()
```

set_palette

The `set_palette()` function in seaborn allows you to specify a color palette for your plots. This can be done by passing in one of the pre-defined seaborn palettes (such as "deep", "muted", "bright", etc.) or by passing in your own custom list of colors or color_palette.

Here is an example of using `set_palette()` to specify the "deep" palette:

You can also pass in a custom list of colors. For example, the following code would set the palette to the colors red, blue, and green:

You can also pass in a number of different arguments to `set_palette`. For example, the following code sets the color palette to a specific hue, with 8 colors, and a desaturated lightness:

Now say we have set palette colors and passed three colors, like we did above and want to plot of 4 or more country line plots?

What color will be assigned to 4th country? Let's see

See it took, color palette we set as - `sns.set_palette("husl",8, .7)`, with eight colors. Even if we are specifying set palette as `['red', 'blue', 'green']`

PYTHON

```
# This will set palette as deep
sns.set_palette('deep')
sns.lineplot(data=tips, x='tip', y='total_bill')
plt.show()
```

PYTHON

```
# Loading different data
import plotly.express as px

gap = px.data.gapminder()
gap.head()
```

seaborn.husl_palette

```
seaborn.husl_palette(n_colors=6, h=0.01, s=0.9, l=0.65, as_cmap=False)
```

Return hues with constant lightness and saturation in the HUSL system.

The hues are evenly sampled along a circular path. The resulting palette will be appropriate for categorical or cyclical data.

The `h`, `l`, and `s` values should be between 0 and 1.

Parameters:

`n_colors` : `int` Number of colors in the palette.

`h` : `float` (0-1) The value of the first hue.

`l` : `float` (0-1) The lightness value.

`s` : `float` (0-1) The saturation intensity.

`as_cmap` : `bool` If True, return a matplotlib colormap object.

`sns.hls_palette()` > This function is similar to `husl_palette()`, but it uses a nonlinear color space that is more perceptually uniform, and saturation in the HLS system.

We can also use 'husl' or 'hsl' parameter in `set_palette` function for the same. Like we did in above example.

PYTHON

```
# Iris data Loading
iris = sns.load_dataset('iris')
```

cubehelix_palette

The `seaborn.cubehelix_palette` function is used to generate a colormap based on the cubehelix color scheme, which is a sequential color map with a linear increase in brightness and a smooth progression through the hues of the spectrum. This function takes several optional parameters such as `start`, `rot`, `gamma`, `light`, `dark`, `reverse` and `as_cmap` to control the properties of the color palette.

For example, the following code generates a cubehelix color palette with 8 colors, starting from a blue hue, and with increasing brightness and a rotation of 0.5:

This palette can be used to color various plotting elements such as bars, lines, and points in a graph.

PYTHON

```
colors = sns.cubehelix_palette(8, start=.5, rot=-.75, gamma=.3, light=.9, dark=.1, reverse=True)
sns.palplot(colors)
```

PYTHON

```
sns.barplot(x='species', y='petal_length', data=iris, palette=colors)
```

Functional tools

map/filter apply functions; reduce aggregates. Often paired with lambdas.

All those function mentioned in coloring have similar implementation.<https://>

Follow below links to read about specif palette - seaborn.pydata.org/api.html#color-palettes

PYTHON

```
sns.heatmap(iris.corr(), cmap=sns.cubehelix_palette(8, start=.5, rot=-.75, gamma=.3, light=.9, dark=.1, as
```

Plotly Express — Interactive Data Visualization

The Plotly Roadmap

Here is the mental map you should keep for the whole session — the three layers of the Plotly stack:

1. **Plotly Go** (`plotly.graph_objects as go`) — the foundational, low-level API. You think in *traces*: `go.Scatter(x=..., y=..., mode='markers')` creates one trace, several traces make the `data`, a `go.Layout(title=..., xaxis=..., yaxis=...)` drives the look, and `go.Figure(data, layout)` assembles the plot. Full pixel-level control, explicit and verbose.
2. **Plotly Express** (`plotly.express as px`) — the high-level, one-line API built *on top of* Go. `px.scatter(df, x=..., y=..., color=..., size=..., hover_name=...)` handles traces, layout and legend automatically and returns a ready figure. Prefer `px` everywhere unless you need Go's low-level machinery (surfaces, contours and subplot grids are Go's turf).
3. **Dash** — the application layer. Dash takes Express/Go figures and wires them into interactive multi-page web apps with dropdowns, sliders and live updates. Dash is out of scope for this notebook but it is the reason Plotly figures embed so naturally into browsers.

Rule of thumb for this session: start in Express (`px`), drop to Go when the chart type is not available in Express (surface, contour, raw circuit layout), and remember Dash exists for anything that must react to user input.

Working with Plotly Go

Visualization

Figures/axes, subplots, encodings (x/y/color). Choose clarity over decoration.

Plotly — interactive plotting

- Plotly = **interactive** charts (hover, zoom)
- Express (`px`) — high-level, charts in a single line
- Go — low-level, trace-level control
- `px.data.tips()/iris()/gapminder()` — built-in datasets
- Roadmap: Go → Express → Dash (apps)

PYTHON

```
# import the libraries
import plotly.graph_objects as go
import numpy as np
import pandas as pd
import plotly.express as px
```

PYTHON

```
# import datasets
tips = px.data.tips()
iris = px.data.iris()
gap = px.data.gapminder()
```

Scatter — go & px

- `go.Scatter` — trace object; join it in `.Figure(trace)`
- `mode='markers' | 'lines' | 'lines+markers'`
- `px.scatter` — color/size/hover in a single call
- `animation_frame` — automatic chart animation

PYTHON

```
trace1 = go.Scatter(x=..., y=..., mode='markers', name=...)
px.scatter(gap, x='lifeExp', y='gdpPercap', color='continent',
           size='pop', hover_name='country',
           animation_frame='year')
```

Deep Dive : Plotly — interactive plots (go vs px)

- Plotly = interactive, zoom, hover, pan web-format graphs
- **go** (graph_objects) low-level: trace + layout + figure
- **px** (express) high-level: one-liners with data semantics (color/size)
- `plotly.express` as `px` — preprocessor for `go`
- modes: markers/lines/markers+lines
- 3 levels: express → go → dash (apps)
- Hover shows point data values
- graphs embed HTML/Jupyter display

- `hover_name` — prominent tooltip label
- `animation_frame` — animate over a column (time!) — killer feature

PYTHON

```
# scatter plot using plotly go

temp_df = gap[gap['year'] == 2007]
temp_df
```

PYTHON

```
trace1 = go.Scatter(x=temp_df['lifeExp'],y=temp_df['gdpPercap'],mode='markers')
trace2 = go.Scatter(x=[0,1,2],y=[0,90,30000],mode='lines')

data = [trace1,trace2]

layout = go.Layout(title='Life Exp Vs GDP per Capita for 2007', xaxis={'title':'Life Exp'},yaxis={'title':
fig = go.Figure(data,layout)

fig.show()
```

Line / Bar

- `px.line` — trends
- `text_auto=True` — value labels
- Stacked/grouped via `barmode`
- Animation bhi bar pe possible

PYTHON

```
px.line(temp_df, x=..., y=...) # time series
px.bar(temp_df, x=..., y=..., text_auto=True) # annotations
px.bar(gap, x='continent', y='pop', color='continent',
       animation_frame='year')
```

Deep Dive : Line & Bar charts

- line: time series for one main column (pop over years)
- multiple countries isin → multi lines compare (color auto)
- bar: aggregations; `text_auto` → annotate values
- grouped of multiple countries via cols selection
- stacked: parameter `barmode='stack'` or title stacking
- `animation_frame 'year'` — animate chart through time!
- Dash-ready interactive controls
- pop composition comparisons
- `px.bar` vertical default; `orientation='h'` horizontal

- `color=` encodes category identity

PYTHON

```
# line plot
# plot india pop line plot
temp_df = gap[gap['country'] == 'India']

px.line(temp_df, x='year', y='pop', title='India pop growth')
```

PYTHON

```
# plot india china pak line plot
temp_df = gap[gap['country'].isin(['India', 'China', 'Pakistan'])].pivot(index='year', columns='country', values='pop')
temp_df
```

Histogram / Pie / Sunburst / Treemap

- `nbins` — histogram bins
- Pie `hole` → donut; `pull` → explode
- Sunburst/treemap — hierarchical data circular/box

PYTHON

```
px.histogram(iris, x='sepal_length', nbins=20)
px.pie(values=..., names=..., hole=0.3) # donut
px.sunburst(tips, path=['sex', 'smoker', 'day', 'time'], values='total_bill')
px.treemap(gap, path=..., values='pop')
```

Deep Dive : Pie / Sunburst / Treemap

- pie — proportion categorical; values vs names pair
- donut: hole param — cleaner center label
- sunburst — hierarchical radial: levels → inner rings outward
- treemap — hierarchical rectangles: size= value, color= measure
- drill-down interactive (click collapse)
- plotly hierarchical charts built-in zoom
- path list = hierarchy chain columns
- values = numeric weight; color adds 2nd dimension
- world pop continent/nation breakdowns
- comparisons share-of-whole readability

PYTHON

```
# Pie -> values -> names
# find the pie chart of pop of european countries in 2007

temp_df = gap[(gap['year'] == 2007) & (gap['continent'] == 'Europe')]

px.pie(temp_df, values='pop', names='country')
```

PYTHON

```
# plot pie chart of world pop in 1952 continent wise -> -> explode(pull)

temp_df = gap[gap['year'] == 1952].groupby('continent')['pop'].sum().reset_index()
px.pie(temp_df, values='pop', names='continent')
```

Heatmap / 3D / Matrix

- Heatmap — pivot table matrix
- `scatter_3d` — 3 interactive dims
- `scatter_matrix` — pairwise scatter grid

PYTHON

```
px.imshow(grid) # heatmap
px.scatter_3d(iris, x=..., y=..., z=..., color='species')
px.scatter_matrix(iris, dimensions=[...]) # pairplot
```

Deep Dive : Heatmap / 3D / Scatter matrix

- `imshow` — 2D matrix color cells (pivot first)
- `scatter_3d` — xy z space colored by categorical — richness
- `scatter_matrix` — all pairs of dimensions — quick correlation eyeball
- `dimensions` list subset limits matrix view
- 3D interactive rotate in plotly
- hover info all dims
- `color_group` by species distinguishes classes
- Express covers all common chart families
- cluster/relationship viz for ML prep
- `px` has `parallel_coordinates` & `dimension`

PYTHON

```
# Heatmap -> find heatmap of all continents with year on avg life exp
#temp_df = tips.pivot_table(index='day', columns='sex', values='total_bill', aggfunc='sum')

temp_df = gap.pivot_table(index='year', columns='continent', values='lifeExp', aggfunc='mean')
px.imshow(temp_df)
```

PYTHON

```
# 3d scatterplot
# plot a 3d scatter plot of all country data for 2007
temp_df = gap[gap['year'] == 2007]
px.scatter_3d(temp_df, x='lifeExp', y='pop', z='gdpPercap', log_y=True, color='continent', hover_name='country')
```

Faceting / Surface / Subplots

- `facet_col/facet_row` — auto subplots
- `facet_col_wrap` — wrap columns
- Surface/contour via `go.Surface` / `go.Contour`
- `make_subplots` — manual layout control

PYTHON

```
px.scatter(tips, x='total_bill', y='tip',
           facet_col='smoker', facet_row='sex')
make_subplots(rows=2, cols=2) # custom grids
fig.add_trace(go.Scatter(...), row=1, col=1)
```

Deep Dive : Faceting & subplots & surfaces

- `facet_col/row` — small multiples (grid panels by category)
- `facetcolwrap` — wrap many facets rows
- `surface/contour go-only` — express gap typedef
- `makesubplots + addtrace` — different charts in grid
- `go` flexibility: any trace combination layout
- Canvas: 3D rotate, zoom hover everywhere
- Story: facet compare & combine dimensions
- `px` doesn't do surface; `go` step down when needed
- professional interactive reports end here

PYTHON

```
# facet plot
tips = px.data.tips()
gap = px.data.gapminder()
```

PYTHON

```
px.scatter(tips, x='total_bill', y='tip', facet_col='smoker', facet_row='sex', color='time')
```

Real-World and AI Engineering Relevance

Where this shows up in AI engineering

- figure, axes and artist model
- plot types and annotations
- statistical and categorical plots
- interactive charts and design principles

EXAM AUR INTERVIEW PERSPECTIVE

Agar koi concept interview ya exam mein poochha jaye, to definition ke saath apna ek chhota practical example zaroor bolo. Sirf theory batane se confidence nahi dikhta - implementation dikhane se dikhta hai.

Quick Reference

Short, representative snippets collected from the unit's notebooks. Use them as syntax reminders, not as complete programs.

PYTHON

PLOTTING WITH MATPLOTLIB

```
# import the library
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

plt.style.use('default')
```

PYTHON

PLOTTING WITH MATPLOTLIB

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

PYTHON

DEEP DIVE : 2D LINE PLOT BASICS

```
# plotting a simple function
price = [48000, 54000, 57000, 49000, 47000, 45000]
year = [2015, 2016, 2017, 2018, 2019, 2020]

plt.plot(year, price)
```

PYTHON

DEEP DIVE : 2D LINE PLOT BASICS

```
# from a pandas dataframe
batsman = pd.read_csv(r"D:\GENAI\01_PREREQUISITS\00_Python\08_Data_Visualization\Datasets\sharma-kohli.csv")
batsman
```

PYTHON

DEEP DIVE : 2D LINE PLOT BASICS

```
# plotting multiple plots
plt.plot(batsman['index'],batsman['V Kohli'])
plt.plot(batsman['index'],batsman['RG Sharma'])
```

PYTHON

DEEP DIVE : 2D LINE PLOT BASICS

```
import matplotlib.pyplot as plt
price = [48000,54000,57000,49000,47000,45000]
years = [2015,2016,2017,2018,2019,2020]
plt.plot(years, price)
```

PYTHON

DEEP DIVE : LABELS, TITLE, COLORS & STYLE

```
# Labels title
plt.plot(batsman['index'],batsman['V Kohli'])
plt.plot(batsman['index'],batsman['RG Sharma'])

plt.title('Rohit Sharma Vs Virat Kohli Career Comparison')
plt.xlabel('Season')
plt.ylabel('Runs Scored')
```

PYTHON

DEEP DIVE : LABELS, TITLE, COLORS & STYLE

```
# colors(hex) and line(width and style) and marker(size)
plt.plot(batsman['index'],batsman['V Kohli'],color='#D9F10F')
plt.plot(batsman['index'],batsman['RG Sharma'],color='#FC00D6')

plt.title('Rohit Sharma Vs Virat Kohli Career Comparison')
plt.xlabel('Season')
plt.ylabel('Runs Scored')
```

Revision Checklist

Work through this list without looking at the notes. Anything you cannot explain aloud belongs back in the revision queue.

- ☐ Plotting with Matplotlib
- ☐ 2D Line plot
- ☐ Pandas fundamentals
- ☐ Deep Dive : 2D Line plot basics
- ☐ Deep Dive : Labels, title, colors & style
- ☐ Deep Dive : Figure show + styles + save
- ☐ Scatter Plots
- ☐ Deep Dive : Scatter plot — correlation
- ☐ Bar chart
- ☐ Bar charts
- ☐ Deep Dive : Bar charts — categorical
- ☐ Histogram
- ☐ Pie charts
- ☐ Histograms
- ☐ Deep Dive : Histogram vs Pie vs Log scale
- ☐ Pie Chart
- ☐ Styles & save
- ☐ Save figure
- ☐ Summary
- ☐ Colored scatterplots

☐ Deep Dive : Colored scatter — 3rd dimension via c

☐ Figure size